

Instituto Tecnológico de Costa Rica

Examen de Ensamblador — Versión convertida a RISC-V RV32I

MT-7003 Microprocesadores y microcontroladores — II Semestre 2025

Propósito de este documento: convertir el enunciado y los archivos de preparación del examen original, que están planteados para ARM/VisUAL2, a RISC-V RV32I, manteniendo los ejercicios y sin resolverlos.

Regla de alcance: la conversión se mantiene dentro de los contenidos indicados del curso: RV32I, registros, load/store, inmediatos, saltos, branches, arreglos/memoria, procedimientos, JAL/JALR, ABI y pila.

Importante: VisUAL2 es un simulador ARM. Para esta versión, el código debe ejecutarse en un simulador RISC-V compatible con RV32I (por ejemplo, Ripes).

Equivalencias usadas

ARM original	RISC-V RV32I	Uso en este examen
R0	x10 (a0)	Primer argumento / resultado de funciones
R1	x11 (a1)	Segundo argumento / registro auxiliar según ejercicio
R2	x12 (a2)	Tercer argumento / temporal según ejercicio
R10	x20 (s4)	Resultado del ejercicio I
R11	x21 (s5)	Segundo resultado del ejercicio I
BL etiqueta	jal ra, etiqueta	Llamada a procedimiento
MOV PC, XX	jalr x0, 0(XX)	Retorno/salto indirecto
STR	sw	Almacenamiento de palabra
LDR	lw	Carga de palabra
[R0,#desplazamiento]	offset(a0)	Direccionamiento base + desplazamiento

Nota sobre R0 → a0

En ARM, R0 es un registro general. En RISC-V, x0 es constante en cero y no puede utilizarse como equivalente de R0. Por ello, para conservar el significado funcional del enunciado, R0 se representa como **a0 (x10)**. De forma análoga, los resultados solicitados en R10 y R11 se representan como **s4 (x20)** y **s5 (x21)**, respectivamente. Esta es una adaptación de nombres/ABI, no una solución de los ejercicios.

Instrucciones generales

Las instrucciones generales del examen ensamblador son las estipuladas en el programa del curso.

A estas se les suman las siguientes instrucciones específicas:

- Usted posee 3 horas para completar el examen.
- El examen se realiza en tríos.
- Adjunto a este enunciado puede encontrar archivos con el “header del ejercicio”. El código ya escrito no puede ser modificado y plantea las condiciones básicas del ejercicio a realizar.

- Modificar este código inicial invalida el ejercicio con una nota de 0.
- Usted debe realizar un diagrama de flujo que explique su algoritmo de solución. Puede hacerse a mano siempre que sea legible.
- Las soluciones deben ser ejecutables en un simulador RISC-V RV32I. En esta versión se sustituye la exigencia original de VisUAL2, que corresponde a ARM.
- Deben subir al TEC Digital un archivo comprimido que contenga una carpeta con solo 6 archivos. De entregar más de 6 archivos se descuentan 10 puntos de la nota final del examen.
- El archivo con la solución de ensamblador: uno por ejercicio.
- El diagrama de flujo: uno por ejercicio.
- La carpeta debe tener los apellidos de los estudiantes.
- Para el diagrama de flujo, si necesita escanear múltiples imágenes puede hacerlo siempre y cuando todas estén en un único documento.
- La hora de entrega final del TEC Digital contempla dificultades de comunicación por el medio virtual; el examen original está pensado para 2 horas y media. Si el TEC Digital no se encuentra disponible, los estudiantes pueden enviar el archivo comprimido al correo del profesor. Entregas tardías implican una nota de 0.
- La defensa del examen se agenda en el Excel indicado en el examen original.
- Las citas poseen una duración de hasta 45 minutos máximo y se realizan mediante Google Meet.
- La solicitud de cita se acepta únicamente por inscripción al Excel.
- En caso de no agendar defensa para las 11:45 pm del 19 de septiembre, se procederá con una calificación binaria de la solución entregada.
- La defensa se basa en explicar los diagramas de flujo, ejecutar el código con la solución o las secciones funcionales y defender el ejercicio correspondiente.

Distribución de puntos

La distribución de puntos por cada ejercicio respeta esta proporción (ejemplo para un ejercicio de 50 puntos):

Componente	Puntos
Diagrama de flujo	10
Comentarios en código	5
Funcionalidad	15
Capacidad de explicar el código	20

Nota: ser totalmente incapaz de explicar el código o el diagrama de flujo implica una nota de 0 en todo el ejercicio.

I. Ejercicio: Conteo de Caracteres — 50 puntos

Considere un espacio de memoria definido. Usando el código ASCII como referencia, usted debe realizar un programa que encuentre los dos caracteres más repetidos en la cadena. El código ASCII del carácter que sale más veces se debe guardar en **s4 (x20)** y el 2.º carácter que sale más veces se debe guardar en **s5 (x21)**.

- Los posibles caracteres son las letras de la A a la Z.
- El registro **a0 (x10)** contiene la dirección inicial del espacio de memoria.
- El registro **a1 (x11)** contiene la dirección final del espacio de memoria.
- Las mayúsculas y minúsculas cuentan para la misma letra. Ej.: en la cadena AxdfghaB la letra A/a aparece dos veces.
- Cada palabra en memoria contiene 4 caracteres ASCII (1 byte por carácter).
- En caso de empate no importa cuál carácter queda en s4 y cuál en s5.
- En caso de que varios caracteres empaten, no importa cuál de todos queda en s4 o s5.
- El archivo header de este ejercicio posee solo una variante del ejercicio; durante la revisión el profesor modificará este header para asegurar todas las variantes.

II. Ejercicio: Ordenar Palabras — 50 puntos

Considere un espacio de memoria que consta de exactamente 10 palabras. En este espacio de memoria se encuentran 10 números en un orden aleatorio. Usted debe realizar un código que escriba en otro espacio de memoria la misma lista de 10 números, pero en orden descendente.

- El registro **a0 (x10)** contiene la dirección inicial del espacio de memoria.
- Los números no se repiten.
- El archivo header de este ejercicio posee solo una variante del ejercicio; durante la revisión el profesor modificará este header para asegurar todas las variantes.

III. Ejercicio: Traducción de Código C — 50 puntos

Para este ejercicio usted debe realizar una traducción del código C del archivo Código_Traduccion_C_a_Esamblador.c a código ensamblador RISC-V RV32I.

- Debe usar **JAL** para realizar el acceso/llamada a funciones y **JALR** para manejar el retorno de funciones.
- El registro **a0 (x10)** debe ser usado para obtener el resultado de todos los métodos.
- La sección "MAIN" del archivo también debe ser traducida a ensamblador.
- Use la convención de llamada de RISC-V correspondiente al material del curso para argumentos, retornos y registros preservados.

Anexo A — Header convertido: Conteo de Caracteres

```
# #####
#     SETUP EJERCICIO CONTEO CARACTERES
# #####

# Definicion de Direcciones de Memoria
li      a0, 0x00FF0000
li      a1, 0x00FF0000
addi    a1, a1, 0x0000000C

# Creacion y almacenamiento de la primera palabra
li      t0, 0x41000000
li      t1, 0x004D0000
add     t0, t0, t1
li      t1, 0x00006100
add     t0, t0, t1
li      t1, 0x0000004F
add     t0, t0, t1
sw      t0, 0(a0)

# Creacion y almacenamiento de la segunda palabra
li      t0, 0x72000000
li      t1, 0x007A0000
add     t0, t0, t1
li      t1, 0x00006100
add     t0, t0, t1
li      t1, 0x0000006D
add     t0, t0, t1
sw      t0, 4(a0)

# Creacion y almacenamiento de la tercera palabra
li      t0, 0x75000000
li      t1, 0x00420000
add     t0, t0, t1
li      t1, 0x00006100
add     t0, t0, t1
li      t1, 0x0000004D
add     t0, t0, t1
sw      t0, 8(a0)

# Creacion y almacenamiento de la cuarta palabra
li      t0, 0x78000000
li      t1, 0x004B0000
add     t0, t0, t1
li      t1, 0x00006100
add     t0, t0, t1
li      t1, 0x00000065
add     t0, t0, t1
sw      t0, 12(a0)

# Reinicio de registros segun necesidades del ejercicio
li      t0, 0

# #####
#     ESPACIO PARA SOLUCION
# #####
```

Nota: este header conserva la estructura y los datos del original; únicamente se adaptan registros, direccionamiento y las instrucciones ARM a RV32I/pseudoinstrucciones del nivel visto.

Anexo B — Header convertido: Ordenar Palabras

```
# #####
#      SETUP EJERCICIO ORDENAR PALABRAS
# #####

# Definicion de Direcciones de Memoria
li      a0, 0x00FF0000

# Creacion y almacenamiento del primer dato
li      t0, 0x00000082
sw      t0, 0(a0)

# Creacion y almacenamiento del segundo dato
li      t0, 0x00000099
sw      t0, 4(a0)

# Creacion y almacenamiento del tercer dato
li      t0, 0x00000077
sw      t0, 8(a0)

# Creacion y almacenamiento del cuarto dato
li      t0, 0x00000005
sw      t0, 12(a0)

# Creacion y almacenamiento del quinto dato
li      t0, 0x00000020
sw      t0, 16(a0)

# Creacion y almacenamiento del sexto dato
li      t0, 0x00000010
sw      t0, 20(a0)

# Creacion y almacenamiento del septimo dato
li      t0, 0x00000001
sw      t0, 24(a0)

# Creacion y almacenamiento del octavo dato
li      t0, 0x00000033
sw      t0, 28(a0)

# Creacion y almacenamiento del noveno dato
li      t0, 0x00000050
sw      t0, 32(a0)

# Creacion y almacenamiento del decimo dato
li      t0, 0x00000063
sw      t0, 36(a0)

# Reinicio de registros segun necesidades del ejercicio
li      t0, 0

# #####
#      ESPACIO PARA SOLUCION
# #####
```

Nota: no se proporciona ninguna parte de la solución del algoritmo de ordenamiento.

Anexo C — Código C del ejercicio de traducción

El código C es independiente de ARM/RISC-V, por lo que el código fuente se conserva sin cambios. Lo que se convierte es su implementación en ensamblador y las reglas de llamada/retorno.

```
// #####
//     UTILS
// #####
// Operación Corrimiento
// param base: Numero a ser desplazado
// param campos: Numero que detalla la cantidad del desplazamiento
int corrimiento(int base, int campos) {
    int resultado = 0;
    resultado = base << campos;
    return resultado;
}

// Selector Operaciones
// param a: Primer parametro
// param b: Segundo parametro
// param op: Operacion a realizar
int operaciones(int a, int b, int op) {
    int resultado = 0;

    if (op == 0) {
        resultado = a + b;
    }
    else if (op == 1) {
        resultado = a&&b;
    }
    else if (op == 2){
        resultado = corrimiento(a, b);
    }
    return resultado;
}

// #####
//     MAIN
// #####
int solucion = 0;
solucion = operaciones(11, 7, 0);
solucion = operaciones(9, 7, 1);
solucion = operaciones(3, 5, 2);
solucion = operaciones(99, 76, 3);
```

Elementos que NO se deben copiar literalmente del examen ARM

- R0 no se convierte en x0: x0 siempre contiene cero. En este documento se usa a0 (x10) para conservar la función de R0.
- R10/R11 se convierten a s4/s5 (x20/x21) para representar los dos resultados solicitados.
- BL se sustituye por JAL con ra como registro de retorno.
- MOV PC, XX se sustituye por JALR x0, 0(XX).
- LDR/STR se sustituyen por LW/SW.
- La sintaxis de direccionamiento ARM [Rn,#offset] se sustituye por offset(Rn).
- La exigencia de VisUAL2 se sustituye por un simulador RISC-V RV32I.

Advertencia de alcance

La conversión anterior usa solamente conceptos que corresponden al material de RISC-V/ensamblador indicado. El examen original exige ARM/VisUAL2, por lo que esas dos partes no pueden mantenerse literalmente al cambiar de ISA.